

Iterative Drift in DeFi Protocols: A Structural Class, a Subclass Theorem, and Static Detection

Anonymous Author(s)

Abstract

A function that is correct for a single execution can become exploitable when applied repeatedly within one transaction. We study this phenomenon in DeFi protocols and identify a class of vulnerabilities we call *iterative drift*, where locally correct operations accumulate bias under composition and violate global invariants.

We introduce a structural typing perspective that distinguishes between local determinism and compositional safety. In particular, we formalize the notion of *TypeE.cumulative* rules: functions that are deterministic and correct at single-call granularity but unsafe under N-step self-composition. For the subclass of quantization-driven rules, we give a sufficient condition for exploitability, combining directional bias, reachability, and bounded callability within a single atomic transaction. This explains real-world exploits such as Bunni v2 (\$8.4M, 2025) and Balancer V2 (\$128M, 2025) within a unified framework.

We implement a static analyzer over Solidity ASTs that detects quantization patterns, propagates drift through call graphs and inheritance, and identifies loop-amplified and unchecked arithmetic variants. Evaluated on a benchmark derived from public post-mortems and synthetic variants, the analyzer detects all instances of the targeted class without false negatives within this benchmark.

Beyond these results, we argue that many security properties verified at single-call granularity implicitly assume compositional stability, which does not hold in general. Making iteration-aware properties explicit is therefore necessary for robust analysis of DeFi protocols.

Keywords

smart contracts, DeFi security, static analysis, iterative drift, program analysis

1 Introduction

A function that is provably correct for any single call can still be exploitable when an attacker chains it many times in one transaction. These vulnerabilities do not arise from incorrect code, but from invariant structures that become visible only under repeated application of an individually correct rule. Existing auditing tools—formal verification, fuzzing, and manual review—operate at single-call granularity and provide no coverage for this class.

DeFi protocols collectively hold hundreds of billions of dollars in user assets, yet suffer a recurring pattern of catastrophic failure that bypasses deployed security infrastructure. In 2023, Euler Finance lost \$197M to an exploit that every auditor who reviewed the contract had missed. In April 2025, Bunni v2 lost \$8.4M to a rounding error that its own developers described as “safe in the context of a single operation.” In November 2025, Balancer V2 lost \$128M to a rounding error in the `_upscaleArray` function—despite

multiple professional audits, Certora formal verification, and active bug bounty programs.

The Balancer case is the clearest evidence for the gap we address. Certora, in their post-exploit analysis, explicitly stated: “*The verified properties did not constrain the relationship between individual swaps or rounding behavior. Scenarios in which iterative operations could systematically increase value due to rounding bias were not ruled out.*” This is a direct admission that the iterative drift class lies outside state-of-the-art formal verification—not because of any failure of execution, but because N-call compositional behavior was not conceived as a verifiable property.

This paper makes four contributions:

- (1) **MST-IR**: A structural intermediate representation classifying protocol rules along four dimensions—execution structure, recursion safety, iteration behavior, and rule classification. The iteration-behavior dimension introduces the *TypeE.internal* / *TypeE.cumulative* distinction as a formal type-level security property.
- (2) **Invariant Boundary Violations (IBV)**: A formalization of the class of exploits where a rule modifies shared state in a way that violates a downstream rule’s invariant assumption.
- (3) **TypeE.cumulative and Iterative Drift (ID)**: A formal characterization of rules that are locally correct at single-call level but exploitable under N sequential applications. For the sub-class of quantization-driven drift rules, we provide: (a) sufficient conditions for exploitability (Theorem 4.1), (b) a proof that $correct(f) \Rightarrow correct(f^N)$ is false for this class, and (c) counterexamples establishing the precise class boundary.
- (4) **AST Analyzer**: A static analyzer on Solidity 0.8.x compact JSON ASTs (§7), with type-resolved call-graph traversals, loop tracking, unchecked arithmetic detection, and drift propagation.

2 Background

2.1 Type Distinctions Used in This Paper

TypeE.internal. A rule is *TypeE.internal* if for every single invocation, it is deterministic and satisfies all stated operational requirements. This is the granularity at which existing auditing tools operate.

TypeE.cumulative. A rule is *TypeE.cumulative* if it is *TypeE.internal*—locally correct per invocation—but there exists N and x such that $f^N(x)$ violates a global invariant. The function is correct at every single step; the invariant violation is only visible under composition.

The analogy from dynamical systems is precise: a function can be locally stable while globally unstable under iteration. “Correct at one step” does not imply “invariant preserved after N steps.” The terms are adapted from prior work [1]; the theorem and analyzer here do not depend on that framework (see §2.3).

Table 1: Comparison with DeFi typing frameworks.

Framework	N-call?	Key difference
CFF [2]	No	Horizontal composition only
SciviK [3]	No	Single-call granularity
Currency flow [4]	No	Information flow, not drift
Certora CVL	No	Explicitly excluded (post-Balancer)
roundme [5]	Partial	Regex only; no call-graph/reachability
MST-IR	Yes	Iteration-aware classification + detection

2.2 Comparison with Existing Frameworks

Table 1 positions MST-IR against existing DeFi security frameworks. None of the prior frameworks distinguish TypeE.internal from TypeE.cumulative or provide N-call iteration-aware detection.

2.3 Role of MST in This Paper

The use of prior work [1] is intentionally minimal. Theorem 4.1 does **not** depend on it, nor does the AST analyzer. MST is used solely as a rule classification layer enabling the TypeE.internal / TypeE.cumulative distinction—a uniform language for comparing hazard classes across protocols. All exploitability results can be stated without it.

3 MST-IR: A Structural Type System

3.1 Iteration Annotations

@TypeE.cumulative (core):

@TypeE.cumulative(variable: V, bias: B, bound: F|none)

bias $\in \{\text{monotonic_decrease}, \text{monotonic_increase}\}$. bound: none $\Rightarrow$ exploitable.

@loop_amplified (v0.4): Applied when quantization ops appear inside a loop body. Condition 3 (callability) is satisfied by loop iteration count. Captures the Balancer V2 pattern.

@unchecked_arithmetic (v0.4): Applied when state variable arithmetic appears inside UncheckedBlock. In Solidity 0.8.x, unchecked subtraction wraps on underflow—a distinct invariant escape path.

3.2 Compiler Detection Rules

```
IF HasQuantization(f) AND HasDirectionalBias(f)
  AND IsRepeatable(f) AND NOT HasCorrection(f)
  THEN EMIT TYPEE_CUMULATIVE [CRITICAL|LOW]
```

```
IF HasQuantization(f) AND QuantizationInLoop(f)
  AND IsRepeatable(f)
  THEN EMIT LOOP_AMPLIFIED
```

```
IF UncheckedStateVarArithmetic(f) AND IsRepeatable(f)
  THEN EMIT UNCHECKED_ARITHMETIC
```

```
% Post-pass: V in drift_vars(CRITICAL) AND unchecked_ops
% THEN EMIT SUPER_CRITICAL
```

Table 2: Condition mapping: theorem $\rightarrow$ compiler predicate $\rightarrow$ AST.

Cond.	Theorem	Predicate	Auto?
1	Local correctness	HasQuantization	✓
2a	Structural drift	HasDirectionalBias	✓
2b	Reachability	(manual)	×
3	Callability	IsRepeatable	✓
4	No correction	NOT HasCorrection	✓ (approx)

4 Invariant Boundary Violations and Iterative Drift

4.1 IBV Sub-classes

We identify four IBV patterns: **(IBV-A) Invariant Asymmetry**—a rule updates one invariant component without updating dependents (Euler Finance, \$197M: donateToReserves adjusts eToken without dToken liabilities); **(IBV-B) Recursive Shared-State Mutation**—a rule iterates with external calls that may re-enter and mutate state; **(IBV-C) Premature Guard Release**—a reentrancy guard released before all invariant-critical updates complete; **(IBV-D) Cross-Boundary State Dependency**—a rule consumes state modified by a prior external call whose invariant has not been re-established, propagating stale state into a downstream computation (v4-stoploss). Each crosses an invariant boundary within a single execution.

4.2 Iterative Drift and TypeE.cumulative

Definition (TypeE.cumulative). f is TypeE.cumulative with respect to invariant I if: (1) $f(x)$ satisfies all operational requirements per invocation; (2) $\exists n, x$ such that $f^n(x) \not\models I$.

Error model. Let $\varepsilon(x) = I(f(x)) - I(x)$. If $\varepsilon(x) \leq -\alpha < 0$ uniformly on reachable $\mathcal{R}$, then the invariant is violated in at most $n^* = \lceil (I(x_0) - I_{\min}) / \alpha \rceil$ steps.

THEOREM 4.1 (ITERATIVE DRIFT INSTABILITY). *Let $f : X \rightarrow X$ be deterministic and I an invariant with threshold $I_{\min}$. Suppose:*

- (1) Local correctness: f satisfies all stated requirements per invocation.
- (2a) Structural drift: $\exists \alpha > 0$ such that $\varepsilon(x) \leq -\alpha$ for all $x \in \mathcal{R}$.
- (2b) Reachability: $\mathcal{R}$ is reachable via protocol-legal operations.
- (3) Bounded callability: f is executable at least n^* times in one atomic transaction—via n^* external calls or a loop of n^* iterations.

Then $\exists n \leq n^$ such that $f^n(x_0) \not\models I$.*

Remark. The loop-based instantiation (Balancer V2) and repeated-call instantiation (Bunni v2) are both covered. Condition 2b is the only condition not automated by the AST analyzer; it requires domain knowledge about reachability of $\mathcal{R}$.

4.3 Conditions: Scope

The four conditions characterize the **quantization-driven sub-class** studied here: within this sub-class, they are jointly sufficient, and the counterexamples in §4.4 show why dropping any one breaks the characterization. Non-quantization TypeE.cumulative lies outside this scope.

4.4 Counterexamples

CE1 (Banker's rounding): $\varepsilon(x)$ takes both signs; no uniform $-\alpha$. Condition 2a fails. **CE2** (Carry compensation): `HasCorrection` = true. **CE3** (Stochastic rounding): non-deterministic; outside scope.

4.5 Primary Case: Bunni v2 (\$8.4M)

```
uint256 newBalance = balance -
    balance.mulDiv(shares, totalSupply);
```

`mulDiv` rounds DOWN. $\varepsilon \in (-1, 0]$ globally. 44 external calls in one flash loan tx; $n_{\text{avg}}^* \approx 48$. Fix: use `mulDivUp`.

4.6 Second Case: Balancer V2 (\$128M)

`_upscaleArray` uses `mulDown` (rounds DOWN) inside `batchSwap` loop. 65+ micro-swaps; Condition 3 satisfied by loop iterations in one transaction. Certora post-mortem confirmed this class was outside existing verification scope.

Total TypeE.cumulative losses, 2025: \$136.4M.

4.7 Security Implication

Within the class defined by Theorem 4.1, $\text{correct}(f) \Rightarrow \text{correct}(f^n)$ is false. All existing tools implicitly assume this implication at single-call granularity.

5 Related Work

Formal verification (Certora, KEVM): per-transaction properties only. Certora post-mortem confirms iterative drift was outside scope.

Fuzzing (Echidna, Medusa): random inputs cannot target adversarial N-call compositions.

Rounding tools. Trail of Bits `roundme` [5]: regex-based per-operation rounding check (Condition 2a only). No call-graph traversal, reachability, or callability analysis.

Fixed-point and numerical error literature. Cumulative rounding error under repeated arithmetic is classical [6]. The distinction of this paper is applying it as a *type-level security property*: (i) Condition 2b and loop-based Condition 3 are not present in numerical analysis; (ii) the `TypeE.internal/TypeE.cumulative` distinction makes the property compiler-detectable; (iii) flash loans compress n^* steps into one atomic transaction—turning slow accumulation into an instantaneous exploit.

6 Cross-Case Unification

IBV and ID are orthogonal: IBV arises from invariant inconsistency within a single execution; ID arises from invariant erosion under repeated execution of a locally consistent rule.

For IBV-pattern cases in the benchmark (P8), the tool signal is `DRIFT_PROPAGATED`: the analyzer exposes a shared-state contamination site rather than a full semantic IBV verdict, flagging functions sharing state with `TypeE.cumulative` sources without requiring downstream invariant semantics.

Table 3: Cross-case comparison.

Case	1-call	N-call	Class	Loss
Euler Finance	✗	✗	IBV-A	\$197M
TakeProfitsHook	✗	✗	IBV-B	—
Silo	Partial	Partial	IBV-C	—
v4-stoploss	Partial	Partial	IBV-D	—
Bunni v2	✓	✗	ID	\$8.4M
Balancer V2	✓	✗	ID+loop	\$128M

Table 4: Tool coverage by hazard class. $\sim^*$ = `roundme` detects rounding direction only (no call-graph/reachability).

Tool	IBV-A	IBV-B	IBV-C	IBV-D	ID
Manual review	$\sim$	$\sim$	$\sim$	✗	✗
Fuzzing	$\sim$	$\sim$	$\sim$	✗	✗
Certora CVL	$\sim$	$\sim$	$\sim$	✗	✗
<code>roundme</code>	✗	✗	✗	✗	$\sim^*$
MST-IR	✓	✓	✓	✓	✓

6.1 Tool Coverage

7 Detection via MST-IR

7.1 AST Analyzer Architecture

`mst_ir_analyzer_ast_v0.4.py` operates in three phases on the Solidity 0.8.x compact JSON AST from `solc -standard-json`.

Phase 1: Indexing. Builds `func_by_id`, `func_by_contract_name`, and `state_vars_by_contract` with parent state vars merged via `baseContracts` resolution.

Phase 2: Per-function analysis. The body walk collects `QuantizationOp` records from `FunctionCall` nodes (via `referencedDeclaration` $\rightarrow$ `ROUNDING_FUNCTIONS` table), `BinaryOperation(/)` and `(*)`, and `YulFunctionCall(div/sdiv/sar)`; each op is tagged `in_loop=True` inside loop ancestors. `IsRepeatable` checks visibility and state mutability; `view/pure` and single-use-modifier functions are excluded. `HasCorrection` combines name heuristics (carry, residual, ...) with structural carry-update detection. Call-graph expansion walks callee bodies recursively; inheritance resolution searches `baseContracts` for callees absent from the current contract.

Phase 3: Drift propagation. Functions modifying a state variable drifted by `TypeE.cumulative` sources are flagged `DRIFT_PROPAGATED`.

7.2 Verdict Logic

```
c1 and bias==DOWN and c3 and c4 -> CRITICAL
c1 and bias==UP and c3 and c4 -> LOW
loop_quantization and repeatable -> LOOP_AMPLIFIED
unchecked_ops and repeatable -> UNCHECKED
% V in drift_vars(CRITICAL) AND unchecked -> SUPER_CRITICAL
```

7.3 Verification on Known Cases

7.3.1 14-Case Benchmark Protocol. Positive set (P1–P8): BunniDirect, BalancerLoop, InternalHelper, InheritanceChain, AssemblyDiv, UncheckedArith, DriftPropagation, IBV_SharedStateAsymmetry.

Table 5: Verification on six known cases.

Function	Expected	Result
BunniLike.withdraw()	CRITICAL	✓
BalancerStyle.batchWithdraw()	CRITICAL + LOOP	✓
CarryCompensated.withdraw()	No fire	✓
AsmVault.redeemAsm()	CRITICAL	✓
Pool.deposit() → _sharesForAssets()	CRITICAL	✓
Child.withdraw() → Base._compute()	CRITICAL	✓

Table 6: Benchmark results. †: verdict on target function. *: path-sensitive FP; see §8.3.

ID	Name	Exp.	Obs.	OK?
P1	BunniDirect	CRITICAL	CRITICAL	✓
P2	BalancerLoop	CRITICAL	CRIT+LOOP	✓
P3	InternalHelper	CRITICAL	CRITICAL	✓
P4	InheritanceChain	CRITICAL	CRITICAL	✓
P5	AssemblyDiv	CRITICAL	CRITICAL	✓
P6	UncheckedArith	UNCHK	UNCHK	✓
P7	DriftPropagation	DRIFT	DRIFT†	✓
P8	IBV_SharedState	DRIFT	DRIFT†	✓
N1–N5	Five neg. cases	NO_FIRE	NO_FIRE	✓
N6	CondCorrection	LOW	CRITICAL	FP*

Table 7: Ablation results.

Toggle	P	R	FN	Notes
Full baseline	1.000	1.000	0	—
no_call_graph	1.000	0.750	2	P3, P4
no_loop	1.000	1.000	0	LOOP annot. lost
no_unchecked	1.000	0.875	1	P6

Negative set (N1–N5): safe patterns. Manual bucket (N6): ConditionalCorrection (path-sensitive, excluded from P/R metrics).

TP=8, FP=0, TN=5, FN=0, manual=1. P=R=F₁=1.000 (benchmark-relative only).

7.3.2 Automation Boundary. HasQuantization and IsRepeatable are fully deterministic. HasCorrection is approximate: conditional-branch correction requires path-sensitive analysis; cross-deployment calls require multi-file input. Soundness claim is scoped to syntactic quantization discovery within the provided compilation unit.

8 Discussion and Limitations

8.1 For Protocol Designers

Round against the protocol: use mulDivUp in withdrawal functions. Loop-based batch operations warrant additional scrutiny; treat @loop_amplified as a design-review trigger.

8.2 For Auditors and Integrators

CRITICAL findings warrant manual verification of Conditions 2b and 3. SUPER_CRITICAL arises when unchecked arithmetic and

TypeE.cumulative drift target the same state variable simultaneously. Integrators should treat DRIFT_PROPAGATED as a shared-risk signal when composing with upstream protocols.

8.3 Limitations

Abstraction level. Flow-insensitive; correction in conditional branches may not be detected (see N6 in Table 6). Reflects chosen abstraction, not a hazard model deficiency.

Invariant specification. IBV invariants not in ASTs; auditor inference required. MST-IR augments rather than replaces invariant reasoning.

Drift scope. Covers quantization-driven drift only. Oracle lag and AMM dynamics require domain-specific modeling.

Correction approximation. HasCorrection may suppress CRITICAL on incorrect inference, or fire CRITICAL when path-sensitive correction exists (N6).

Cross-contract. Single compilation unit; multi-file projects need all sources together.

Benchmark scope. Synthetic but case-grounded; P/R are benchmark-relative.

Economic attacks. MEV, governance, oracle manipulation outside structural typing scope.

These limitations reflect a deliberate design choice: MST-IR targets a specific, previously under-typed class and provides a tractable AST-level detection method.

9 Conclusion

DeFi exploits arise not only from faulty code, but from mis-specified boundaries of determinism across execution and time. A function can be provably correct for every individual call and still be exploitable under adversarial N-call composition within a single transaction.

This paper identified two structurally distinct vulnerability classes—IBV and Iterative Drift—that together explain catastrophic failures including the \$8.4M Bunni v2 and \$128M Balancer V2 incidents. TypeE.cumulative formalizes the failure mode where a locally correct rule becomes unsafe under N-call self-composition. Theorem 4.1 gives sufficient conditions for exploitability in the quantization-driven sub-class, and counterexamples establish the class boundary.

The AST analyzer correctly classifies all quantization-driven patterns with zero false negatives on a 14-case benchmark. The TypeE.internal / TypeE.cumulative distinction suggests a template: whenever a property is verified at single-call granularity, ask whether its N-call self-composition is safe under adversarial callability. Extending this to oracle lag, non-linear AMM dynamics, and cross-protocol composition is a natural direction for future work.

References

- [1] Anonymous. Omitted for blind review. 2026.
- [2] K. Babel, P. Daian, M. Kelkar, A. Juels. Clockwork Finance: Automated Analysis of Economic Security in DeFi. *IEEE S&P* 2023.
- [3] I. Choi et al. SciviK: A Versatile Framework for Specifying and Verifying Smart Contracts. *OOPSLA* 2021.
- [4] L. Aceto, S. Lybech. A Type System for Currency Flow in DeFi Protocols. *ECOOP* 2024.

- [5] Trail of Bits. roundme: A tool for detecting rounding errors in Solidity. 2023. <https://github.com/trailofbits/roundme>.
- [6] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.
- [7] Certora. Post-exploit analysis: Balancer V2 invariant violation. November 2025.

A Open Science

The artifacts for this paper—the AST analyzer (`mst_ir_analyzer_ast_v0.1.py`) and benchmark suite (`mst_ir_benchmark.py`)—are available at: <https://anonymous.4open.science/r/mst-0E22/>

The analyzer requires Python 3.9+ and `solc` (Solidity compiler). The benchmark requires `npx solc` accessible in `PATH`. Example analyzer outputs for the Bunni and Balancer cases are included in the supplementary material.

B Ethical Considerations

This paper studies a class of DeFi vulnerabilities (iterative drift) and presents a static analyzer for detecting them. We address the following ethical dimensions:

Use of vulnerability data. All case studies (Euler Finance, Bunni v2, Balancer V2) are based on publicly disclosed post-mortem reports and documented exploits. No new vulnerabilities were discovered or disclosed in this work. No private or unpublished information was used.

Dual-use considerations. The static analyzer and benchmark are designed for defensive purposes: detecting vulnerable patterns before deployment. The detection predicates (`HasQuantization`, `IsRepeatable`, `HasDirectionalBias`) identify structural patterns, not exploit payloads. The tool does not generate attack transactions or interact with live protocols. We release artifacts to enable auditors and protocol designers to identify and remediate vulnerable code.

Responsible handling of benchmark cases. The benchmark contracts are synthetic reproductions of documented exploit patterns, not copies of production code. They do not contain private keys, live addresses, or deployable attack contracts.

Limitations as an ethical safeguard. As stated in §8.3, the analyzer is flow-insensitive and scope-limited to syntactic quantization patterns. It does not replace manual auditing or formal verification, reducing the risk of over-reliance on automated verdicts.